Hardening Email Sender Authentication

Exploiting SPF, DKIM, and DMARC — and Building a Defence

Contents

1 Overview

Email remains one of the most widely abused communication channels for impersonation and phishing attacks. Over the last two decades, three authentication protocols have been standardised to combat email sender spoofing: **SPF** (Sender Policy Framework), **DKIM** (DomainKeys Identified Mail), and **DMARC** (Domain-based Message Authentication, Reporting and Conformance). Despite their widespread adoption, real-world deployments are riddled with misconfigurations and implementation weaknesses that allow a determined attacker to bypass all three layers simultaneously — a class of vulnerability captured by OWASP A02:2025 (Security Misconfiguration) and A07:2025 (Authentication Failures).

This lab has three complementary objectives:

1. **Attack.** Students will exploit deliberate misconfigurations in a controlled email infrastructure to deliver spoofed messages that pass (or are not rejected by) SPF, DKIM, and DMARC checks.
2. **Detect.** Students will train and evaluate a machine-learning classifier that analyses email headers and metadata to flag spoofed messages, providing a last line of defence when protocol-level checks fail.
3. **Harden.** Students will fix the underlying DNS and MTA misconfigurations and verify that the previous attacks are blocked, practising defence-in-depth at the protocol layer.

Topics covered.

- How SPF, DKIM, and DMARC work and how they interact.
- Common misconfigurations that enable spoofing (soft-fail SPF, missing DKIM, DMARC `p=none`, relaxed alignment).
- Direct MTA attacks using `swaks` and `espoofer`.
- Header-injection and envelope/header `From:` mismatch attacks.
- Feature engineering from email headers for ML-based detection.
- Evaluating a classifier against single- and multi-layer spoofing patterns.
- Hardening DNS zones and the receiving MTA to close the gaps.

Readings.

- RFC 7208 (SPF), RFC 6376 (DKIM), RFC 7489 (DMARC).
- J. Chen, V. Paxson, J. Jiang — *Composition Kills: A Case Study of Email Sender Authentication*, USENIX Security 2020 (included in `containers_setup/espoofer/papers/`).
- `https://github.com/chenjj/espoofer` — `espoofer` tool documentation.

Lab Environment. All tasks run inside a Docker Compose network (`net-10.9.0.0/24`) that you will set up using the provided scripts. No external Internet access is required for the attack tasks. A host machine with Docker and Docker Compose installed is sufficient. This lab has been developed and tested on the official SEED Labs Ubuntu 20.04 virtual machine.

2 Background: How SPF, DKIM, and DMARC Work

Before attacking the system, you should understand what each protocol checks — and just as importantly, what it does *not* check.

2.1 SPF (Sender Policy Framework)

SPF lets a domain owner publish, via a DNS TXT record, the list of IP addresses authorised to send mail on behalf of that domain. On message receipt, the receiver checks whether the connecting IP is in the SPF record of the domain found in the **envelope sender** (SMTP `MAIL FROM`, also called the `Return-Path`).

Key limitation: SPF only checks the *envelope* sender. The `From:` header that the recipient actually sees in their mail client is *never* validated by SPF on its own. An attacker can therefore use any envelope sender they like (e.g. from their own SPF-authorised domain) while forging the visible `From:` to impersonate someone else.

Typical SPF record qualifiers:

- `+all` — pass everyone (essentially no protection).
- `~all` — *soft-fail*: receivers should accept but mark suspicious (commonly silently ignored).
- `-all` — *hard-fail*: receivers should reject.
- `?all` — neutral, no opinion.

2.2 DKIM (DomainKeys Identified Mail)

DKIM adds a `DKIM-Signature:` header to outbound messages that cryptographically signs a selection of headers and the body. The public key is published in DNS at `selector._domainkey.example.com`. Receivers fetch the key, recompute the signature, and check that the signing domain (`d=`) matches and the signature is valid.

Key limitation: a `dkim=fail` result, by itself, does not cause rejection at most receivers — it is just an annotation. Only DMARC turns DKIM failure into an enforcement action. Also, the DKIM signing domain (`d=`) need not match the visible `From:` domain; that linkage is enforced only by DMARC *alignment*.

2.3 DMARC (Domain-based Message Authentication, Reporting and Conformance)

DMARC sits on top of SPF and DKIM and adds two crucial concepts:

1. **Alignment.** DMARC requires that the domain authenticated by SPF *or* DKIM matches the visible `From:` domain. Alignment can be *relaxed* (`aspf=r`, `adkim=r`; same organisational domain) or *strict* (`aspf=s`, `adkim=s`; exact match).
2. **Policy.** The DMARC record declares what receivers should do with messages that fail alignment: `p=none` (monitor), `p=quarantine` (spam folder), or `p=reject` (refuse delivery).

Key limitation: DMARC with `p=none` is effectively monitor-only; spoofed messages are still delivered. Many domains publish `p=none` "for reporting" and never tighten it, leaving the door open.

2.4 Composition Weaknesses

Even with all three protocols deployed, attackers can exploit *composition* flaws between MTAs — for example, by sending multiple `From:` headers, by using header-injection through encoded fields, or by abusing differences in how the envelope and header are parsed at different hops. The `espoofer` tool catalogues many such attacks; you will use it in Task 2.

3 Lab Environment

3.1 Container Architecture

The lab uses four containers connected on the `10.9.0.0/24` subnet. Figure 1 and Table 1 summarise each container's role.

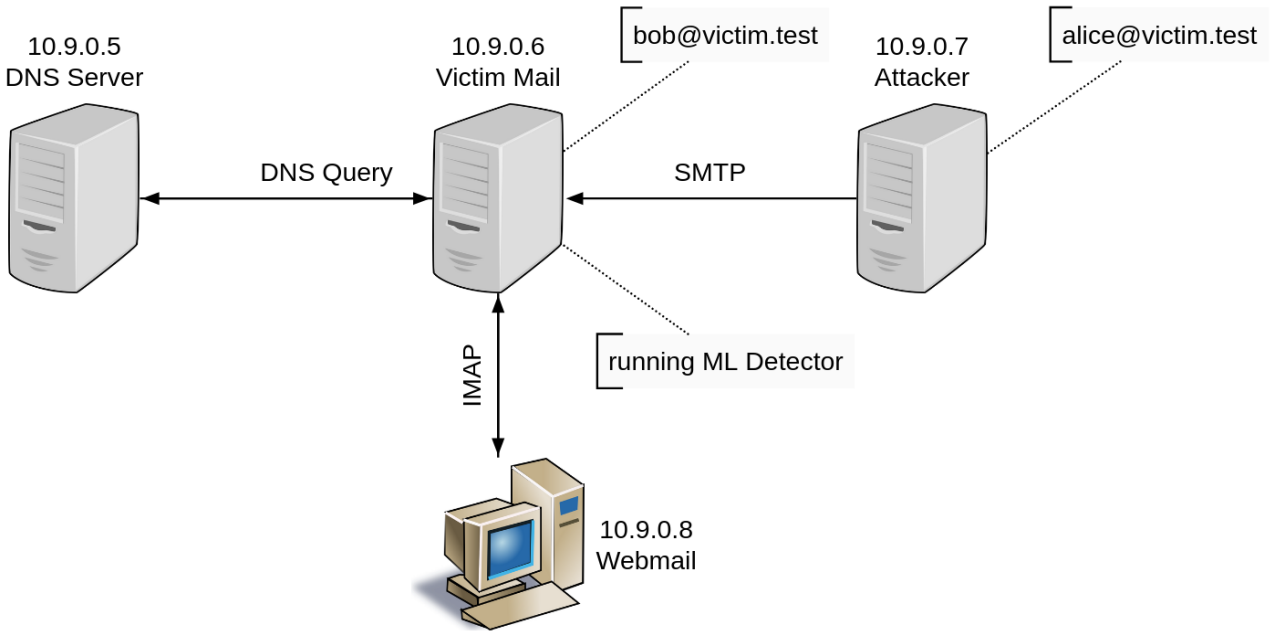

Figure 1: Docker Container Network deployed in this lab.

Table 1: Docker containers and their roles

Container	IP	Image	Role
`dns-server-10.9.0.5`	`10.9.0.5`	Custom BIND9	Authoritative DNS for `victim.test` and `attacker.test` zones.
`victim-mail-10.9.0.6`	`10.9.0.6`	`docker-mailserver`	Postfix/Dovecot mail server for `victim.test`; runs SPF, DKIM, and DMARC validation.
`attacker-sender-10.9.0.7`	`10.9.0.7`	`seed-ubuntu:large`	Attack workstation with `swaks` and `espoofer` pre-installed.
`webmail-10.9.0.8`	`10.9.0.8`	Roundcube (or similar)	Browser-based webmail client for `victim.test`; lets students inspect delivered mail in a familiar UI.

DNS zones. Two DNS zones are served by the BIND9 container:

- `victim.test` — the target domain, configured with intentional weaknesses (SPF soft-fail `~all`, no DKIM selector published, DMARC policy `p=none`). The exact misconfiguration is changed

task by task in `containers_setup/bind/zones/db.victim.test`.

- `attacker.test` — the attacker's domain, with a valid SPF record authorising `10.9.0.7`, a published DKIM public key, and a permissive DMARC record. This lets the attacker container send mail that itself authenticates correctly — which is precisely what enables several of the bypass techniques.

Mail accounts. The victim mail server hosts the following accounts:

```
alice@victim.test   password: pass123
bob@victim.test     password: 123pass
user@victim.test    (hashed, for postfix-accounts.cf)
```

Exposed ports. From the host machine you can reach the victim mail server on the following local ports:

```
SMTP:  localhost:2525  ->  victim-mail:25
IMAP:  localhost:1143  ->  victim-mail:143
HTTP:  localhost:8080  ->  webmail:80    (webmail UI)
```

3.2 Setup and Commands

Step 1 — Start the containers. From the root of the `containers_setup` directory:

```
$ dcdown                  # bring any running containers down first
$ dcup -d                 # start all four containers detached
$ dockps                  # verify all containers are running
```

The aliases `dcup`, `dcdown`, `dockps`, and `docksh` are provided by the standard SEED Labs Ubuntu 20.04 image. If you are running on a vanilla Ubuntu host, substitute `docker compose up -d`, `docker compose down`, and `docker ps` respectively.

Step 2 — Run the first-time initialisation script. Open a shell inside the attacker container and run the setup script *once*:

```
$ docksh attacker-sender-10.9.0.7
(attacker)$ cd /home/seed/init_setup
(attacker)$ bash run-me-1st.sh
```

This script updates the system, installs `swaks`, installs the Python dependencies required by `espoofer`, and clones the `espoofer` repository into `/home/seed/espoofer`.

Step 3 — Verify DNS resolution. Still inside the attacker container, confirm that the custom DNS server is resolving both zones correctly:

```
(attacker)$ dig MX  victim.test               @10.9.0.5
(attacker)$ dig TXT victim.test               @10.9.0.5
(attacker)$ dig TXT _dmarc.victim.test        @10.9.0.5
(attacker)$ dig TXT attacker.test             @10.9.0.5
```

Reloading the DNS server. Whenever you edit a zone file under `containers_setup/bind/zones/`, you must reload BIND so the new records take effect. A helper script is provided:

```
$ ./containers_setup/bind/reload-zones.sh
# or directly:
$ docker exec dns-server-10.9.0.5 rndc reload
```

Stopping the environment. When you are finished:

```
$ dcdown    # stops and removes the containers (volumes are preserved)
```

4 Task 1: Per-Protocol Configurations and `swaks` Attacks

The sender-authentication behaviour for `victim.test` is controlled by DNS records defined in the BIND zone file `containers_setup/bind/zones/db.victim.test`.

By commenting and uncommenting specific lines in this file, you can simulate four deployment scenarios in increasing order of protection:

1. no sender authentication at all,
2. SPF only (initially with soft-fail `~all`),
3. DKIM only,
4. DMARC only (illustrative — DMARC needs SPF or DKIM to be useful).

In each sub-task below, you will:

- edit `db.victim.test` to obtain the desired configuration,
- reload the DNS server and verify the records using `dig`,
- craft a `swaks` command that sends a spoofed email under that configuration,
- retrieve the message and analyse the authentication-related headers.

You should both run the commands directly in the shell *and* encode your final attack commands into the provided `/home/seed/attacks/attack.sh` script so that you can easily repeat each attack during Task 3 (data collection for the ML classifier).

Bump the SOA serial. Each time you edit `db.victim.test`, remember to increment the serial number on the SOA line before reloading BIND, otherwise BIND may not actually pick up your changes.

4.1 Task 1.1 — No Authentication

In this baseline scenario, `victim.test` publishes *no* SPF, DKIM, or DMARC records: receivers have no protocol-level information about who is allowed to send mail for this domain.

Configure. Open `containers_setup/bind/zones/db.victim.test` and ensure all three records (SPF, DKIM, DMARC) are commented out. Bump the SOA serial and run `./reload-zones.sh`. Verify with `spoofmaster> policy` that all three sections return empty answers.

```
spoofmaster> spf
Target Email:    bob@victim.test
Real Sender:     attacker@victim.test
Fake Sender:     ceo@victim.test
Message Body:    Hi Bob, urgent request from the CEO...
```

Attack.

Inspect. Open Bob's inbox in the webmail UI (http://localhost:8080) and examine the full headers of the delivered message.

Questions.

1. Which (if any) `Authentication-Results` fields are present for SPF, DKIM, or DMARC?
2. Does the message reach Bob's inbox? What evidence in the headers (if any) would allow a human or filter to detect the spoofing?
3. How difficult would it be for an attacker to impersonate *any* user at `victim.test` in this configuration?

4.2 Task 1.2 — SPF-Only Deployment

In this scenario, `victim.test` publishes an SPF record but no DKIM or DMARC records. You will compare soft-fail (`~all`) and hard-fail (`-all`) variants to understand why the qualifier matters.

Configure (soft-fail). In `db.victim.test`, uncomment the soft-fail SPF line:

```
@   IN  TXT  "v=spf1 mx ~all"
```

Leave DKIM and DMARC commented. Bump the serial, reload, verify with `spoofmaster> policy`.

Attack. This time the attacker uses an envelope sender from their own SPF-authorised domain (`attacker.test`) while forging the visible `From:` to look like Alice at `victim.test`:

```
spoofmaster> spf
Target Email:    bob@victim.test
Real Sender:     attacker@attacker.test    # envelope from attacker.test
Fake Sender:     alice@victim.test         # forged visible From:
Message Body:    Hi Bob, here's that report you asked for.
```

Optional — compare with hard-fail. Switch the active SPF line in the zone to `-all`, reload, and re-run the same `spoofmaster> spf` flow. Compare delivery outcomes and the `Received-SPF` header value.

Questions.

1. What is the `spf=` result for the message? Which domain did SPF authenticate against?
2. What domain appears in the visible `From:` header that Bob sees, and how does it relate to the SPF-authenticated domain?
3. Does SPF, by itself, prevent Bob from being fooled? Explain in terms of envelope vs. header sender.
4. How does switching from `~all` to `-all` change the outcome? Is the SPF result alone enough to block this attack class?

4.3 Task 1.3 — DKIM-Only Deployment

In this scenario, `victim.test` publishes a DKIM key but no SPF or DMARC records. Receivers can verify a cryptographic signature on the message, but no policy ties that result to a delivery decision.

Configure. A DKIM key pair has already been generated for the victim domain (see `maildata/opendkim/keys/vic` Uncomment the `mail._domainkey` TXT record in `db.victim.test` so the public half is published in DNS. Keep SPF and DMARC commented. Bump the serial, reload, and verify with `spoofmaster> policy`.

A legitimate test message from `alice@victim.test` to `bob@victim.test` (sent through the webmail UI) should now appear in Bob's inbox with `dkim=pass` in `Authentication-Results`.

Attack. The attacker sends a spoofed message carrying a syntactically valid but cryptographically bogus `DKIM-Signature` header (constructed automatically by `attack.sh`'s `dkim` mode):

```
spoofmaster> dkim
Target Email:    bob@victim.test
Real Sender:     attacker@attacker.test
Fake Sender:     ceo@victim.test
Message Body:    Approve the wire transfer per attached invoice.
```

Note on DKIM failure modes. Depending on the verifier implementation, you may observe `reason="signature verification failed"` (the textbook outcome — the verifier fetched the public key and the cryptographic check failed) or `reason="key not found in DNS"` (an implementation quirk in the OpenDKIM build shipped with `docker-mailserver`, where the verifier rejects the key internally even though DNS returned it correctly). Both are forms of `dkim=fail` and produce the same downstream behaviour. Record which reason your verifier produced.

Questions.

1. What is the `dkim=` result for the spoofed message?
2. Does the message still reach Bob's inbox? Why might a receiver accept a message with `dkim=fail`, regardless of the specific failure reason?
3. What additional policy or configuration would be required to reject messages with broken DKIM signatures?
4. In a real-world attack, an attacker spoofing a domain they do not control cannot generate a valid signature with the victim's private key, and cannot publish a public key under the victim's domain. The forged DKIM header in this task simulates both limitations. Which of these does the DKIM protocol itself address, and which is purely operational?

4.4 Task 1.4 — DMARC-Only Deployment

DMARC enforces *alignment* of authenticated identities (from SPF or DKIM) with the visible `From:` domain. If a domain publishes DMARC but does not deploy SPF or DKIM, DMARC has no underlying authentication results to align — and the policy becomes effectively meaningless. This task makes that clear.

Configure. In `db.victim.test`, uncomment only the DMARC line:

```
_dmarc IN TXT "v=DMARC1; p=none; aspf=r; adkim=r"
```

Leave SPF and DKIM commented out. Bump the serial, reload, verify with `spoofmaster> policy`.

Attack. The attacker sends from their own SPF-authorised domain (`attacker.test`) with a forged visible `From:` header claiming `victim.test`. With no SPF or DKIM records published for `victim.test`, there is nothing for DMARC to align against:

```
spoofmaster> dmarc
Target Email:    bob@victim.test
Real Sender:     attacker@attacker.test    # envelope from attacker.test
Fake Sender:     ceo@victim.test           # forged visible From:
Message Body:    DMARC is published but has nothing to enforce on.
```

Questions.

1. What are the `spf=`, `dkim=`, and `dmarc=` results in `Authentication-Results`? Why does `dmarc=` report what it does?
2. DMARC's main mechanism is *alignment* between an authenticated identity (from SPF or DKIM) and the visible `From:` domain. What identity does DMARC have available to align in this scenario?
3. In Task 4.3 you will deploy SPF + DMARC together. Predict what the `dmarc=` result will be for this same attack when SPF is also active. Why?

5 Task 2: Multi-Layer Bypass with `espoofer`

In Task 1 you exploited each protocol (SPF, DKIM, DMARC) in isolation using `swaks`. In this task, you will use the `espoofer` tool to craft attacks that combine multiple bypass techniques and misconfigurations at once.

Before starting Task 2, restore the *combined misconfiguration* state of `db.victim.test` — soft-fail SPF, no DKIM selector, and DMARC `p=none`:

```
; --- COMBINED MISCONFIGURATION (for Task 2) ---
@       IN  TXT     "v=spf1 mx ~all"
; default._domainkey is intentionally absent
_dmarc  IN  TXT     "v=DMARC1; p=none; aspf=r; adkim=r"
```

Reload BIND, then continue.

5.1 Task 2.1 — Exploring Available Attack Cases

`espoofer` is a research tool that implements a catalogue of known email-sender-authentication bypass techniques discovered by Chen, Paxson, and Jiang (USENIX Security 2020). It operates in three modes: *server*, *client*, and *manual*. In this lab you use *server* mode, where `espoofer` acts as the sending MTA.

```
(attacker)$ cd /home/seed/espoofer
(attacker)$ python3 espoofer.py -l
```

This lists all built-in attack cases. Note the case identifiers (e.g. `server_a1`, `server_a2`, ...).

Questions.

1. How many distinct server-mode attack cases are available?
2. Pick three cases and describe in your own words which authentication property (SPF, DKIM, DMARC, or their composition) each one exploits.

5.2 Task 2.2 — Configuring `espoofer`

Edit `/home/seed/espoofer/config.py` to target the lab infrastructure:

```
config = {
    "attacker_site":           b"attacker.test",
    "legitimate_site_address": b"admin@victim.test",  # spoofed From:
    "victim_address":          b"bob@victim.test",    # target inbox
    "case_id":                 b"server_a1",          # change to test others

    "server_mode": {
        "recv_mail_server":      "10.9.0.6",
        "recv_mail_server_port": 25,
        "starttls":              False,
    },
}
```

5.3 Task 2.3 — Running Combined Attacks

Run `espoofer` using the configured case:

```
(attacker)$ python3 espoofer.py           # uses config.py settings
# OR override the case ID on the command line:
(attacker)$ python3 espoofer.py -id server_a2
```

For each chosen case, retrieve the message from Bob's inbox and examine the `Authentication-Results` header. Save the raw email (`.eml`) to disk — you will need these messages as the *multi-layer attack* subset of the dataset in Task 3.

Questions (repeat for at least three different case IDs).

1. For each case, record the `spf=`, `dkim=`, and `dmarc=` results.

2. Which cases result in `dmarc=pass` even when something suspicious is happening with SPF or DKIM?
3. What does the `From:` header look like to Bob in each case?
4. Relate each successful case back to the misconfigurations you exploited in Task 1 (SPF, DKIM, and DMARC).
5. Explain, in protocol terms, why combining multiple weaknesses (a "cocktail" of bypasses) is more effective than any single technique alone.

5.4 Task 2.4 — Optional: Manual-Mode Attack

Use manual mode to craft a raw spoofed message with full control over every header field:

```
(attacker)$ python3 espoofer.py -m m \
  -helo attacker.test \
  -mfrom attacker@attacker.test \
  -rcptto bob@victim.test \
  -ip 10.9.0.6 -port 25 \
  -data $'From: Alice <alice@victim.test>\r\nSubject: Spoofing\r\n\r\nBody here.'
```

Question. Explain what each flag controls at the SMTP protocol level (`HELO`, `MAIL FROM`, `RCPT TO`, `DATA`...) and how it relates to SPF, DKIM, or DMARC verification.

6 Task 3: ML-Based Spoofing Detection

In this task, you will build and evaluate a machine-learning classifier that distinguishes spoofed emails from legitimate ones based on header features. You will specifically compare how well the classifier performs against *single-protocol* bypasses (Task 1) versus *multi-layer* `espoofer` attacks (Task 2).

6.1 Task 3.1 — Dataset Preparation

Collect raw email headers from:

- legitimate emails (e.g. messages sent between `alice@victim.test` and `bob@victim.test` when SPF, DKIM, and DMARC are all configured correctly — you can reuse the legitimate test from Task 1.3),
- single-protocol bypass attacks using `swaks` (Tasks 1.1–1.4),
- multi-layer bypass attacks using `espoofer` (Task 2, at least three case IDs).

For each email, extract features such as:

- `spf_result` — encoded as `pass=0, softfail=1, fail=2, none=3`;
- `dkim_result` — `pass=0, fail=1, none=2`;
- `dmarc_result` — `pass=0, fail=1, none=2`;
- `from_return_path_mismatch` — Boolean (1 if the domain in `From:` differs from `Return-Path:`);
- `helo_from_mismatch` — Boolean (1 if the HELO domain differs from the `From:` domain);
- `num_received_hops` — count of `Received:` headers;
- `has_dkim_signature` — Boolean;
- `reply_to_different_domain` — Boolean;
- other header-derived features you consider useful.

Because the lab-generated dataset will be small, you should *supplement* it with a publicly available labelled corpus (e.g. SpamAssassin, CEAS 2008) to improve class balance and statistical significance.

Feature extraction hint. Python's standard `email` library can parse raw messages:

```
import email

with open("raw_message.eml", "r") as f:
    msg = email.message_from_file(f)

from_addr     = msg.get("From", "")
return_path   = msg.get("Return-Path", "")
auth_results  = msg.get("Authentication-Results", "")
received_hdrs = msg.get_all("Received", [])
```

6.2 Task 3.2 — Training a Baseline Classifier

Train at least one `scikit-learn` classifier (e.g. Random Forest, Logistic Regression, SVM, Gradient Boosting) to predict whether an email is *legitimate* or *spoofed* based on your feature set. Use an 80/20 train/test split with stratified 5-fold cross-validation.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import classification_report, confusion_matrix

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

clf = RandomForestClassifier(n_estimators=100, random_state=42)
clf.fit(X_train, y_train)

y_pred = clf.predict(X_test)
print(classification_report(y_test, y_pred,
      target_names=["Legitimate", "Spoofed"]))
print(confusion_matrix(y_test, y_pred))
```

Report precision, recall, F1-score, and accuracy for each classifier you train.

Lab Scope This task is a bit outside of the scope of this lab but this is the mitigation mechanism chosen to detect spoofed emails and this protect clients. The classifiers (`RandomForest` and `SVM`) are alredy implemented and all that needs to be done in this task is to enter the victim container with an interactive shell and run the `RUNME-1st.sh` script to install the necessary python dependencies. After that run the `detector.py` script to see the email classifications. This being said you are still encouraged to come up with your own detection model.

6.3 Task 3.3 — Single vs Multi-Layer Bypass Evaluation

To understand how well your classifier handles different kinds of attacks, evaluate it separately on:

- the subset of **single-protocol** bypass emails (Tasks 1.1–1.4), and
- the subset of **multi-layer** `espoofer` emails (Task 2).

Compute and compare metrics (precision, recall, F1-score) on these two subsets.

Questions.

1. Is your classifier more effective at detecting single-protocol bypasses or multi-layer `espoofer` attacks? Explain why in terms of the feature distributions.
2. Which features have the highest importance scores (for tree-based models) or largest coefficients (for linear models)? Does this match your intuition from Tasks 1 and 2?
3. What is the cost of a *false negative* (a spoofed email classified as legitimate) versus a *false positive* (a legitimate email flagged as spoofed) in a real deployment? How should this asymmetry affect your choice of decision threshold?
4. Describe at least two ways an attacker could adapt their spoofing strategy to evade your classifier.
5. What additional features or data sources would you recommend adding in a production-grade system (e.g. content features, sender reputation, behavioural signals, time-of-day patterns)?

7 Task 4: Hardening the Victim Domain

Having successfully attacked the `victim.test` domain and built an ML detector that flags suspicious headers, this final task asks you to fix the underlying misconfigurations at the protocol layer. You will then re-run the attacks from Tasks 1 and 2 to verify that the hardening is effective.

7.1 Task 4.1 — SPF Hardening

Edit the BIND zone file `containers_setup/bind/zones/db.victim.test` and change the SPF record from soft-fail to hard-fail:

```
; Before (vulnerable):
@   IN  TXT  "v=spf1 mx ~all"

; After (hardened):
@   IN  TXT  "v=spf1 mx -all"
```

Bump the zone serial and reload BIND, then verify the new record is published:

```
$ docker exec dns-server-10.9.0.5 rndc reload
(attacker)$ dig +short TXT victim.test @10.9.0.5
# expect: "v=spf1 mx -all"
```

Two attack variants. To understand precisely what hard-fail SPF buys you, run *two* attacks against the hardened zone and compare results.

Variant A — Envelope = `attacker.test`, header `From:` = `victim.test`. This is the Task 1.2 attack, unchanged. The attacker uses their own SPF-authorised domain in the envelope while forging the visible `From:` header.

```
(attacker)$ swaks \
  --from attacker@attacker.test \
  --to bob@victim.test \
  --server 10.9.0.6 --port 25 \
  --helo mail.attacker.test \
  --header "From: CEO <ceo@victim.test>" \
  --header "Subject: Variant A --- envelope from attacker.test" \
  --body "Hi Bob, hard-fail SPF on victim.test has no effect on this attack."
```

Variant B — Envelope = `victim.test`, header `From:` = `victim.test`. This is the naive attack where the attacker also claims `victim.test` in the SMTP envelope. It is what hard-fail SPF is actually designed to stop.

```
(attacker)$ swaks \
  --from ceo@victim.test \
  --to bob@victim.test \
  --server 10.9.0.6 --port 25 \
  --helo mail.attacker.test \
  --header "From: CEO <ceo@victim.test>" \
  --header "Subject: Variant B --- envelope from victim.test" \
  --body "Hi Bob, this envelope claims victim.test from an unauthorised IP."
```

For each variant, retrieve the message (if it was delivered) and inspect the `Received-SPF` and `Authentication-Result` headers, or, if the message was rejected at SMTP, capture the rejection code from the `swaks` output.

Predicted results. Variant A: `spf=pass` (for `attacker.test`), message delivered. Variant B: `spf=fail` under `-all`, message rejected at SMTP (compare to `spf=softfail` and delivery under `~all`).

Questions.

1. For each variant, what domain does SPF actually authenticate against? Look at the `Received-SPF` header (or the rejection reason) to confirm.
2. Why does hard-fail SPF on `victim.test` have no effect on Variant A, no matter how strict the qualifier?
3. Why does it block Variant B?
4. State, in one sentence, the structural limitation of SPF that Variant A exploits.
5. What protocol-level mechanism would be required to block Variant A? (You will deploy it in Tasks 4.2 and 4.3.)

Key takeaway. Tightening SPF from soft-fail to hard-fail closes the door on attackers who claim your domain in the envelope, but it does *nothing* against attackers who use a different envelope domain while forging the visible `From:`. SPF is by design blind to the header `From:`; closing that gap requires DMARC alignment, which you will configure in Task 4.3.

7.2 Task 4.2 — Publishing a DKIM Key

Hard-fail SPF (Task 4.1) closes the naive envelope attack but leaves the envelope/header mismatch attack untouched. The next layer of defence is DKIM: by signing legitimate outbound mail with a

private key whose public counterpart is published in DNS, the victim domain gives receivers a way to cryptographically verify that a message actually originated from the claimed sender's infrastructure.

Step 1 — Generate the key pair. If you did not already keep the key pair from Task 1.3, regenerate it on the attacker container or host:

```
$ openssl genrsa -out dkim_private.pem 2048
$ openssl rsa -in dkim_private.pem -pubout -out dkim_public.pem

# Extract the single-line base64 public key (no PEM headers)
$ grep -v '^-' dkim_public.pem | tr -d '\n'
```

Step 2 — Publish the DKIM selector. Add the following record to `db.victim.test` (replace `<PUBKEY>` with the base64 string from Step 1):

```
default._domainkey IN TXT "v=DKIM1; k=rsa; p=<PUBKEY>"
```

Bump the zone serial, reload BIND, and verify:

```
$ docker exec dns-server-10.9.0.5 rndc reload
(attacker)$ dig +short TXT default._domainkey.victim.test @10.9.0.5
# expect: "v=DKIM1; k=rsa; p=MIIBIj..."
```

Step 3 — Configure the victim MTA to sign outbound mail. Configure Postfix and OpenDKIM on the victim mail server to sign outgoing messages from `victim.test` with the private key from Step 1, using the selector `default`. The specific OpenDKIM configuration files (`KeyTable`, `SigningTable`, `TrustedHosts`) are documented in the project `README`.

Send a legitimate test message from the webmail UI as `alice@victim.test` to `bob@victim.test`, open Bob's inbox, and confirm that the message now carries a valid `DKIM-Signature:` header and that `Authentication-Results` reports `dkim=pass header.d=victim.test`. If you instead see `dkim=none` or `dkim=permerror` on this *legitimate* test, the MTA is not signing correctly — fix this before proceeding, otherwise every subsequent observation is meaningless.

Step 4 — Re-run the attacks and observe the results. The point of DKIM is to make tampered or unsigned mail *detectable*. To see what DKIM does (and does not) catch, re-run two attack families against the now-DKIM-signing victim domain:

Attack 1 — Single-protocol bypass with `swaks` (from Task 1.2 / 4.1, Variant A). The envelope/header mismatch attack, unchanged:

```
(attacker)$ swaks \
  --from attacker@attacker.test \
  --to bob@victim.test \
  --server 10.9.0.6 --port 25 \
  --helo mail.attacker.test \
  --header "From: CEO <ceo@victim.test>" \
  --header "Subject: Post-DKIM swaks test" \
  --body "Same attack as Task 1.2, now against a DKIM-publishing victim."
```

Attack 2 — Multi-layer bypass with `espoofer` (from Task 2.3). Re-run at least three of the `espoofer` server-mode cases that succeeded in Task 2.3 (for example `server_a8`, `server_a11`, `server_a14`):

```
(attacker)$ cd /home/seed/espoofer
(attacker)$ python3 espoofer.py -id server_a8
(attacker)$ python3 espoofer.py -id server_a11
(attacker)$ python3 espoofer.py -id server_a14
```

For each delivered message, capture the `Authentication-Results` header and record the `spf=`, `dkim=`, and `dmarc=` verdicts side by side with the corresponding results from Tasks 1.2 and 2.3.

Predicted results. *Attack 1 (swaks):* `dkim=none` — the attacker did not include any `DKIM-Signature` header, and DKIM does not require one. Message still delivered, because nothing yet enforces "unsigned mail from `victim.test` is suspicious."

Attack 2 (espoofer): most cases now show `dkim=fail` or `dkim=none`, but the message is still delivered. Some composition cases (e.g. multiple-`From` attacks) may still produce `dkim=pass` for `attacker.test` because the receiver verified the attacker's signature against the "wrong" `From` header.

Questions.

1. Compare the `Authentication-Results` headers before and after publishing the DKIM key, for both Attack 1 and Attack 2. Tabulate the `spf=`, `dkim=`, and `dmarc=` verdicts for each scenario.
2. Why is Attack 1 (`swaks`) not blocked by publishing a DKIM key, even though `victim.test` now has a working signing infrastructure? In particular, what header is the attacker not required to include?
3. For each `espoofer` case you re-ran, classify it as "blocked," "detected but still delivered," or "undetected and delivered." Which composition attacks (if any) still produce `dkim=pass` against the hardened domain, and why?
4. State the structural limitation of DKIM that the surviving attacks exploit. What additional mechanism would be required to turn `dkim=fail` (or `dkim=none` on a domain that should be signing) into a delivery decision rather than an annotation?

Key takeaway. Publishing a DKIM key gives receivers a way to *verify* signatures but, on its own, does not stop a single message from being delivered. A missing or broken signature is just an annotation in `Authentication-Results` — not a rejection. The mechanism that turns these annotations into enforcement is DMARC, which you will configure in Task 4.3.

7.3 Task 4.3 — DMARC Policy Enforcement

Update the DMARC record to move from monitoring (`p=none`) to enforcement, and tighten alignment from relaxed to strict:

```
; Quarantine policy (intermediate step towards full enforcement):
_dmarc  IN  TXT  "v=DMARC1; p=quarantine; pct=100; aspf=s; adkim=s;
↪  rua=mailto:dmarc@victim.test"

; Or full rejection:
_dmarc  IN  TXT  "v=DMARC1; p=reject;     pct=100; aspf=s; adkim=s;
↪  rua=mailto:dmarc@victim.test"
```

Note the change from `aspf=r` (relaxed) to `aspf=s` (strict) alignment, which requires the `From:` domain to *exactly* match the SPF or DKIM authenticated domain (not just share an organisational suffix).

Questions.

1. Re-run the attacks from Tasks 1 and 2 with the new DMARC policy. Which attacks are now blocked, and which (if any) still succeed?
2. What is the operational risk of moving directly to `p=reject` without first running in `p=none` (and analysing the `rua` reports) for a period of weeks?
3. Summarise the three-protocol hardening as a defence-in-depth diagram showing which protocol catches which attack class. A hand-drawn sketch included in your report is acceptable.

7.4 Task 4.4 — Defence in Depth: Combining ML and Protocol Hardening

With the victim domain now hardened, re-feed a few of the surviving attack messages (if any) into your Task 3 classifier.

Questions.

1. Does the ML classifier add value on top of a correctly configured SPF/DKIM/DMARC stack? Where, and why?
2. Identify one attack from Task 2 that protocol hardening blocks and one that the ML classifier would have flagged independently.

8 Submission

Students should submit a lab report in PDF format containing:

1. **Environment verification** — screenshots of the four running containers (`docker compose ps`) and successful DNS resolution for both zones.
2. **Task answers** — all numbered questions from Tasks 1 through 4 answered with supporting evidence (terminal output, header excerpts, screenshots).
3. **Attack logs** — the contents of `spf-spoof-attack.log` and the relevant `Authentication-Results` headers for at least three `espoofer` cases.
4. **ML results** — the full `classification_report` output, a confusion matrix, the feature importance plot (or coefficient table), and a comparison of single-protocol vs multi-layer evaluation metrics.

5. **Hardening evidence** — before/after zone-file excerpts and re-run attack outputs showing the effect of each hardening step (SPF, DKIM, DMARC).
6. **Reflection** — a short paragraph (200–300 words) discussing the limitations of both protocol-level defences and the ML classifier, and what a production-grade solution would look like (e.g. ARC, BIMI, MTA-STS, sender reputation).

Academic integrity notice. All attacks in this lab must be conducted exclusively within the provided Docker network. Do *not* attempt any of these techniques against real email infrastructure or third-party domains. The `victim.test` and `attacker.test` TLDs are reserved by RFC 2606 and cannot resolve on the public Internet.

9 References

[1] OWASP Foundation, *A02:2025 — Security Misconfiguration*, https://owasp.org/Top10/2025/A02_2025-Security_Misconfiguration/.
[2] OWASP Foundation, *A07:2025 — Authentication Failures*, https://owasp.org/Top10/2025/A07_2025-Authentication_Failures/.
[3] S. Kitterman, *Sender Policy Framework (SPF) for Authorizing Use of Domains in Email*, RFC 7208, IETF, 2014.
[4] D. Crocker, T. Hansen, M. Kucherawy, *DomainKeys Identified Mail (DKIM) Signatures*, RFC 6376, IETF, 2011.
[5] M. Kucherawy, E. Zwicky, *Domain-based Message Authentication, Reporting, and Conformance (DMARC)*, RFC 7489, IETF, 2015.
[6] J. Chen, V. Paxson, J. Jiang, *Composition Kills: A Case Study of Email Sender Authentication*, USENIX Security Symposium, 2020. https://www.usenix.org/conference/usenixsecurity20/presentation/chen-jianjun
[7] J. Chen, *espoofer: An Email Spoofing Testing Tool*, https://github.com/chenjj/espoofer.
[8] DMARCLY, *How to Implement DMARC/DKIM/SPF to Stop Email Spoofing/Phishing: The Definitive Guide*, https://dmarcly.com/blog/how-to-implement-dmarc-dkim-spf-to-stop-email-spoofing-phishing-th 2024.
[9] Infraforge, *SPF, DKIM, DMARC: Common Setup Mistakes*, https://www.infraforge.ai/blog/spf-dkim-dmarc-common-setup-mistakes, 2026.
[10] Roobal, R. Saxena, V. Dillu, *Real-Time Email Spoofing Detection Using Machine Learning and Timestamp Anomaly Analysis*, JATIT, 2025.

10 Acknowledgements

- The `espoofer` tool is developed by Jianjun Chen, Vern Paxson, and Jian Jiang and is available at https://github.com/chenjj/espoofer.
- The container infrastructure (DNS, mail server, attacker workstation, and webmail) was developed as part of the course project by Guilherme, João, Ayrton, and Ana at FEUP.
- The lab format is inspired by the SEED Security Lab series by Wenliang Du, Syracuse University (https://seedsecuritylabs.org).